

Shuri's Last Attempt

MIRAGE 2.0 — Reverse Engineering — Walkthrough

CATEGORY	DIFFICULTY	PORT
Reverse Engineering	Hard	1710/tcp
FILES	VULNERABILITY	STATUS
shuri_last_attempt (stripped, PIE)	N/A — custom bytecode VM reversal	Solved

Step 1 — First Contact

“The core logic is buried behind its own execution layer” is the load-bearing sentence in this challenge's brief — read literally, it means the real check isn't ordinary compiled code at all, it's an interpreter running some separate program. Connecting shows three options: Request diagnostic trace, Submit recovery sequence, Abort connection.

Option 1 (“request diagnostic trace”) prints a chunk of hex-looking “microcode.” Before spending time decoding it, it's worth checking whether anything in the program's control flow ever reads it back or transforms it further — if nothing does, it's flavor text, not a clue. That check comes up empty here: the trace blob is printed once and never referenced again anywhere in the binary. It's a decoy, and the challenge's own hint (“not every path through the core leads to the real answer”) all but says so directly.

Step 2 — Finding the Real Execution Layer

Option 2 asks for a “recovery sequence” and, unlike a normal string comparison, the disassembly around it doesn't look like ordinary validation code — instead there's a tight loop that reads a byte, uses it to index into a jump table, and dispatches to one of many small handler blocks before looping again. That's the signature shape of a fetch-decode-execute interpreter: the input isn't being compared directly at all, it's being fed as data into a small virtual machine, and the real logic lives in whatever program that VM is running — an embedded byte array elsewhere in the binary.

Reconstructing the opcode handlers one at a time (by reading what each jump table entry's target block actually does) recovers a small instruction set:

```
0x10  load next key character
0x21  XOR accumulator with an embedded constant
0x22  ADD an embedded constant (mod 256)
0x31  compare accumulator, roll it into a running checksum
0x41  no-op
0x70  finalize / compare the running checksum
0x7f  halt
```

Reading the embedded program itself (rather than just the opcode handlers) shows it repeats the Load → XOR → ADD → compare block exactly 19 times before the final checksum opcode — which is how the required input length (19 characters) is discovered, well before ever needing to solve for the actual characters.

Step 3 — The Same Trap as Before: Anti-Debug

As in the Mind Stone challenge, two gates run before the VM does anything useful: a `/proc/self/status` `TracerPid` check, and a `CLOCK_MONOTONIC_RAW`-bracketed ~120,000-iteration LCG spin with a 25ms budget. Tripping either diverts to a branch that prints a plausible-looking flag — confirmed fake here the same way as before, by deliberately tracing the process and observing that a distinct “watched” branch fires:

```
MIRAGE{th3_v4ult_b3l0ngs_t0_th4n0s} <-- decoy, only shown if traced
```

The lesson carried over directly from the Mind Stone challenge: reverse this one entirely statically, and only ever test candidate answers against a completely untraced, natively-run process.

Step 4 — Solving the VM's Checksum Offline

Because each of the 19 positions runs the identical `XOR → ADD → compare-and-roll` sequence against its own embedded constants, the per-position arithmetic can be inverted algebraically one character at a time — no brute force needed, just re-implementing the VM's own math backward in Python using the constants extracted from the embedded program, then checking the result against the final rolling-checksum target from the `0x70` opcode.

Step 5 — Running It

```
$ nc <host> 1710
SHURI@CORE> 2
[INPUT] Recovery sequence: SHURI-PRIME-AXIOM-7
```

FLAG

```
MIRAGE{shur1_f1n4l_4tt3mpt_br0ke_th3_vm!}
```

Takeaway

“The logic is buried behind its own execution layer” is worth taking literally in any reversing challenge — a comparison loop that reads opcode-shaped bytes and dispatches through a table is not implementing the check directly, it's implementing an interpreter for the check. Reconstructing the small instruction set first, then reading the embedded program as its own separate artifact, is far more tractable than trying to trace the interpreter's execution byte by byte.